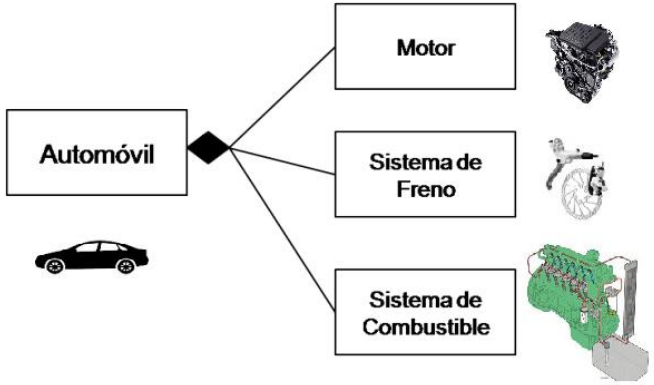
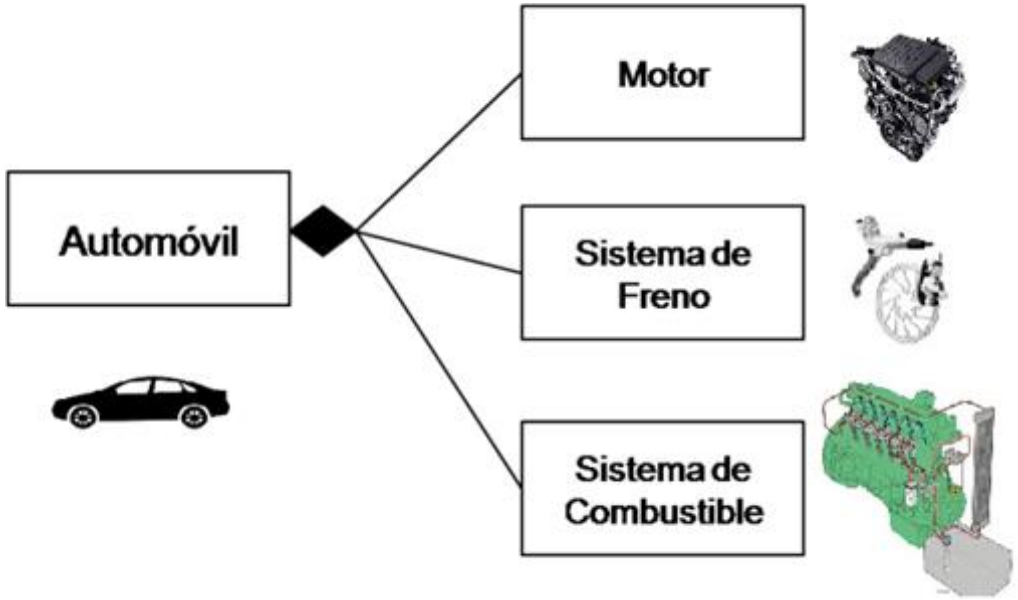
Composición



¿Qué es la composición?

LA **COMPOSICIÓN** es una técnica de diseño en programación orientada a objetos (POO) que permite construir objetos complejos a partir de otros más simples



¿Cómo funciona la composición?

Se definen clases para los componentes individuales

Se crea una clase contenedor para contener instancias de los componentes

Se delegan tareas para lograr la funcionalidad deseada

¿Qué ventajas tiene la composición?

Permite una mayor flexibilidad, modularidad y reutilización

→ Permite cambiar o reemplazar fácilmente los componentes de un objeto sin afectar a su funcionalidad o identidad.

→ Reduce la duplicación de código y los errores

→ Permite crear objetos complejos a partir de objetos más simples

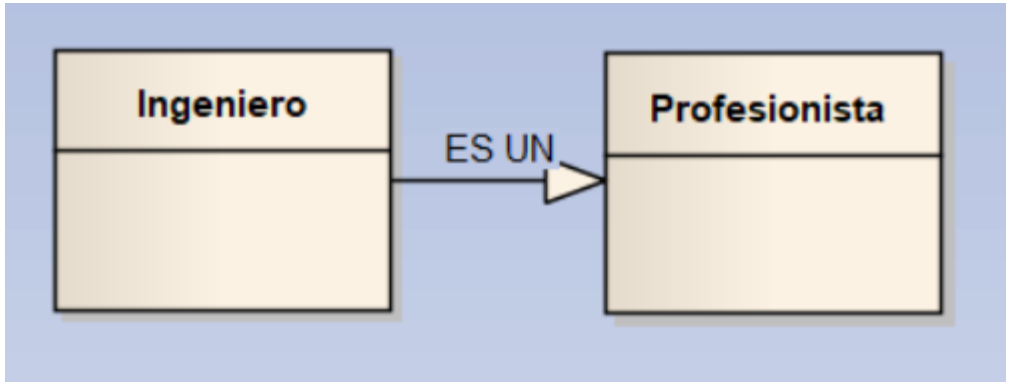
¿Cómo se implementa la composición?

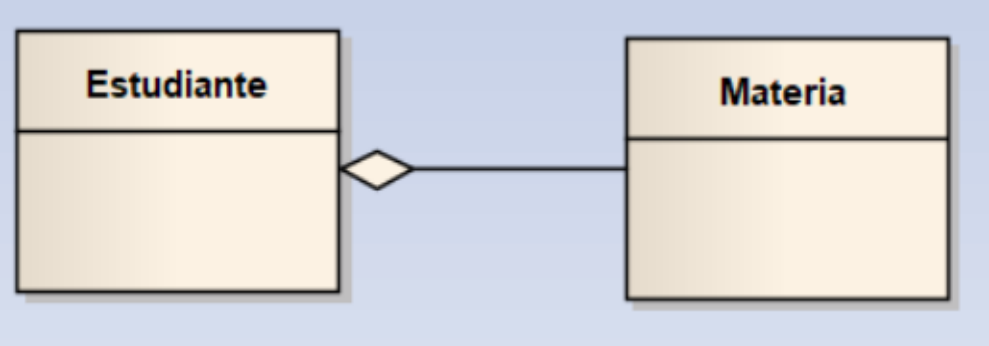
Mediante el uso de variables de instancia de otros objetos

En algunos lenguajes existe el patrón de diseño composite o algunos referentes a lo mismo

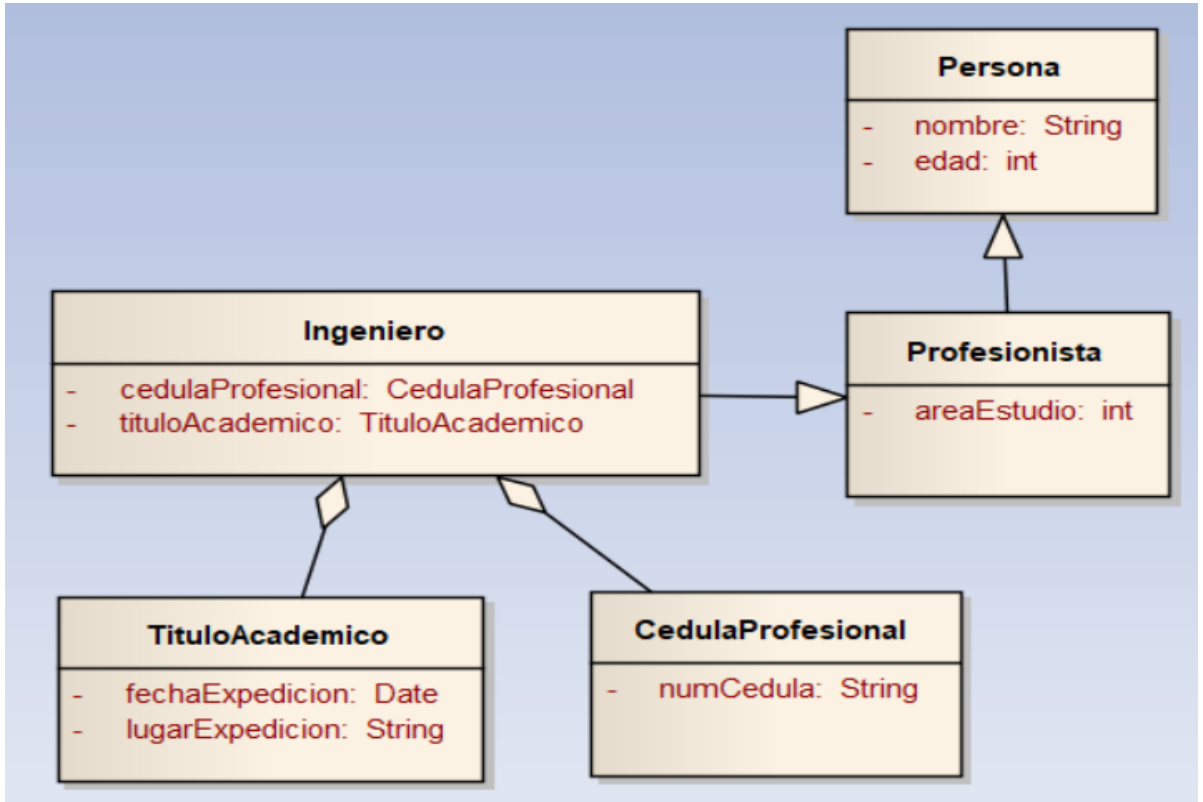
¿Cuál es la diferencia entre la composición y la herencia?

La herencia representa una relación “ES UN” y la composición representa una relación “TIENE UN”





¿Cuál es la diferencia entre la composición y la herencia?



Asociación y Agregación

Asociación:

Cuando la relación entre los objetos es “normal”

es la relación entre dos o más objetos que están conectados entre sí. Es una conexión que permite que los objetos se conozcan o interactúen

Agregación:

Cuando la relación entre los objetos es “fuerte”

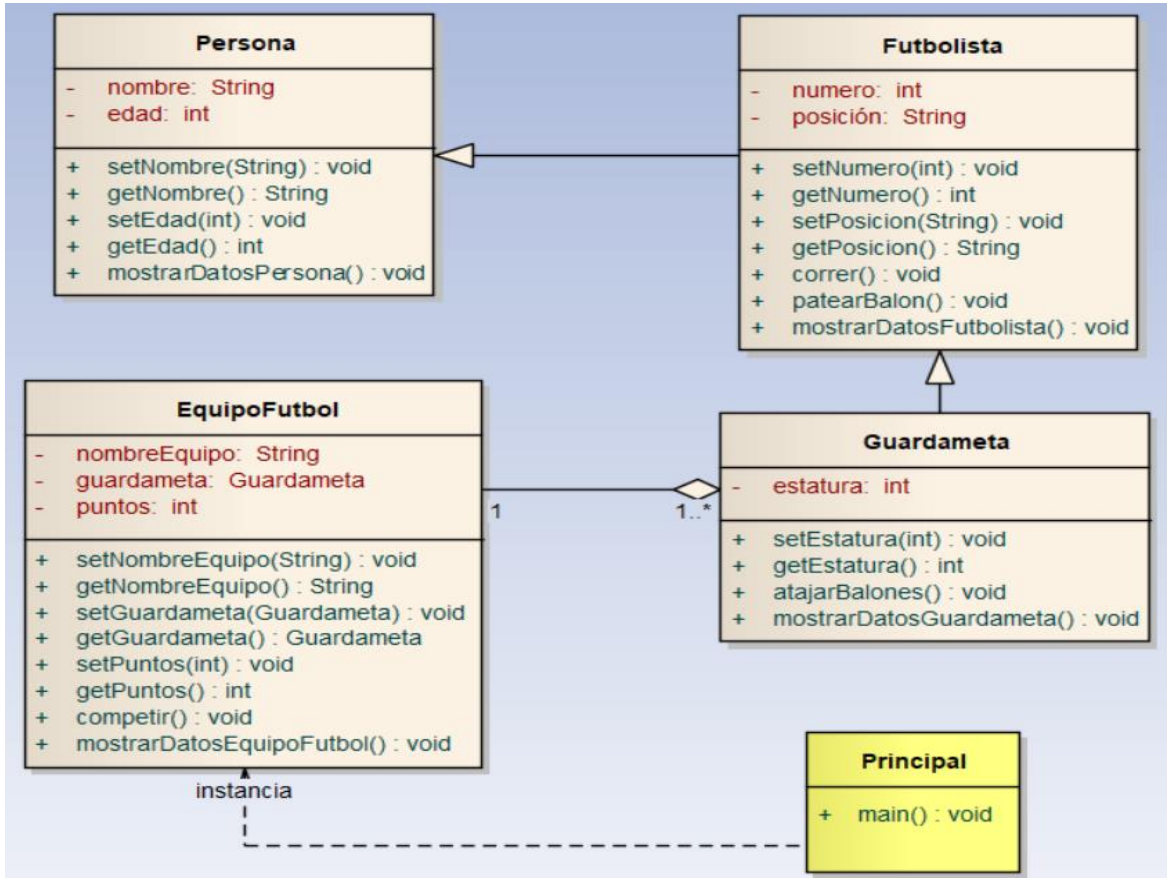
Es una relación entre clases que indica que una clase contiene a otra. Es un tipo de asociación que se muestra como una relación "tiene un

Asociación y Agregación

Asociación:

Agregación:

Composición



Código del diagrama anterior

```
internal class Persona
{
    String nombre;
    int edad;

    1 referencia
    public string Nombre { get => nombre; set => nombre = value; }
    1 referencia
    public int Edad { get => edad; set => edad = value; }
    0 referencias
    public String mostrarDatos()
    {
        String datos = "Nombre " + Nombre;
        datos += "Edad: " + Edad;
        return datos;
    }
}
}
```

Herencia

```
internal class Futbolista:Persona
{
    private int numero;
    private string posicion;

    1 referencia
    public int Numero { get => numero; set => numero = value; }
    1 referencia
    public string Posicion { get => posicion; set => posicion = value; }
    0 referencias
    public string correr() { return "Corriendo"; }
    0 referencias
    public string patearBalon() { return "Pateando balon"; }
    0 referencias
    public string mostrarDatosFutbolista() {
        string datos = "Numero " + Numero;
        datos += "Posicion: " + Posicion;
        return datos;
    }
}
}
```

Código del diagrama anterior

```
internal class Guardameta:Futbolista
{
    private int estatura;

    1 referencia
    public int Estatura { get => estatura; set => estatura = value; }
    0 referencias
    public string atajarBalonesU() { return "Atajando balones"; }
    0 referencias
    public String mostrarDatosGuardameta() {
        String datos;
        datos = "Estatura: " + Estatura;
        return datos;
    }
}
}
```

```
internal class EquipoFutbol
{
    private string nombreEquipo;
    Guardameta guardameta;
    private int puntos;

    1 referencia
    public string NombreEquipo { get => nombreEquipo; set => nombreEquipo = value; }
    1 referencia
    public int Puntos { get => puntos; set => puntos = value; }
    0 referencias
    internal Guardameta Guardameta { get => guardameta; set => guardameta = value; }
    0 referencias
    public string competir() { return "Compitiendo"; }
    0 referencias
    public string mostrarDatosEquipo()
    {
        string datos = "Nombre Equipo" + NombreEquipo;
        datos += guardameta.mostrarDatosGuardameta();
        datos += "Puntos: " + Puntos;
        return datos;
    }
}
}
```

Composición

Código del diagrama anterior